



RS2000 UHF RFID reader module

I am looking for the price for RS2000 UHF RFID reader module

Thought for 8s

For **RS2000 / Impinj R2000-chip UHF RFID reader module**, the current public market price is roughly:

Type	Price range	Notes
Generic RS2000/R2000 UHF RFID reader module	US\$30–54 / piece	Alibaba supplier listing, often MOQ 1 piece.
WINNIX Impinj RS2000/R2000 UHF RFID module	Usually quote-based	Specs show 840–960 MHz, 30 dBm, ISO18000-6C/EPC C1G2, up to 24 m reading range with 12 dBi antenna. Price is not clearly shown publicly.
4-port long-range R2000-chip reader/module	Usually quote-based / higher price	More complete reader hardware, likely more expensive than bare module.

A practical buying budget:

Bare RS2000/R2000 UHF RFID module: about **US\$30–60 each**

Better industrial module / known brand: about **US\$80–150 each**

Complete fixed reader using R2000 chip, multi-port: often **US\$150–400+ each**

For your sourcing, I would treat **US\$30–54** as the low-cost China module price, but ask supplier to confirm: **chip is real Impinj R2000/RS2000**, output power **30 dBm**, frequency region **US/EU/China**, interface **UART/USB/RS232/TTL**, and whether **antenna is included**.

For negotiation, use this target price:

Quantity	Target price
1 sample	US\$35–60 / pc
10 pcs	US\$30–50 / pc
50 pcs	US\$25–45 / pc
100 pcs+	US\$20–40 / pc

Before ordering, ask for a **datasheet photo**, **module label photo**, and **test video reading multiple tags**. Some sellers say “R2000/RS2000” but may use a cheaper clone chip, so confirmation matters.

give me the datasheet for Generic RS2000/R2000 UHF RFID reader module

I'll look for a public datasheet-style spec for the generic RS2000/R2000 module. Since “generic” modules vary by supplier, I'll separate confirmed common specs from items you must verify with the seller.

Thought for 6s

Yes. For **Generic RS2000/R2000 UHF RFID reader module**, the closest public datasheets are usually based on the **Impinj Indy RS2000 / R2000** platform or China-made R2000 module datasheets. The exact datasheet depends on supplier, but this is the typical technical datasheet content.

Generic RS2000/R2000 UHF RFID Reader Module Datasheet Summary

1. Product Description

Product name: RS2000 / R2000 UHF RFID Reader Module
Chip/platform: Usually Impinj Indy R2000 / RS2000-compatible module
RFID type: UHF RFID reader/writer module
Application: Fixed RFID reader, handheld reader, gate reader, warehouse reader, production line reader, access control, logistics, asset tracking.

The official **Impinj Indy RS2000** is described as a fully integrated RAIN RFID reader module based on the Indy R2000 reader chip. It is designed as an embedded solution for adding UHF RFID reader capability into other devices.

2. Main Specifications

Item	Typical specification
Frequency	840–960 MHz global UHF band
Common regions	865–868 MHz EU, 902–928 MHz US, 920–925 MHz China , customizable
Protocol	EPC C1G2 / ISO18000-6C , sometimes ISO18000-6B
RF output power	Usually 0–30 dBm adjustable

Antenna port	1-port, 4-port, 8-port, or 16-port depending on model
Interface	UART TTL, RS232, USB, RS485, Ethernet depending on module board
Reading distance	Usually 5–15 m , depends heavily on antenna, tag, power, and environment
Multi-tag reading	Supported
Frequency mode	FHSS or fixed frequency
SDK	Usually C#, C++, Java, demo software, serial command protocol
Power supply	Commonly 5V / 9V / 12V depending on module board
Working temperature	Often around -20°C to +60°C or similar
Use case	Embedded RFID reader, industrial reader, long-range reader

Some recent R2000 module datasheets list support for **EPC C1G2 / ISO18000-6C, ISO18000-6B, 865–868 MHz / 902–928 MHz**, and **FHSS or fixed-frequency transmission**.

3. Example Module Variants

Module type	Description
1-port R2000 module	Basic embedded module for single antenna systems
4-port R2000 module	Common for small fixed readers
8-port R2000 module	Used for warehouse, cabinet, production line, multi-antenna systems
16-port R2000 module	Used for large multi-antenna industrial systems

For example, one public 8-port R2000 module datasheet lists model **RRU2889M**, size **67.5 × 51 × 7.9 mm**, and weight **75 g**. A 16-port version lists size **117 × 92 × 8 mm** and weight **100 g**.

4. Typical RF Performance

Parameter	Typical value
Output power	Up to 30 dBm
Power adjustment	Software adjustable
Read range	Around 5–15 m , sometimes more with 12 dBi antenna
Tag reading speed	Depends on firmware; some modules advertise hundreds of tags/sec
Tag protocol	EPC Gen2 / ISO18000-6C
RSSI support	Often supported
Dense reader mode	Depends on firmware/module

Important: the **reading distance is not only module-dependent**. It depends on antenna gain, tag type, cable loss, environment, mounting angle, and legal RF power limit in your country. Tiny detail, big headache — classic RFID.

5. Interface / Communication

Generic modules usually support one or more of these:

Interface	Usage
UART TTL	Embedded board integration
RS232	Industrial controller / PC connection
USB	PC demo/test connection
RS485	Industrial long-distance communication
Ethernet	Complete fixed reader version, not always bare module

Some suppliers provide a separate **R2000 UHF RFID Reader Protocol** document for serial commands, tag inventory, read/write operations, kill/lock functions, and memory access.

6. Antenna Information

Item	Typical
Connector	SMA female / MMCX / IPEX depending on module
Antenna impedance	50Ω
Recommended antenna	6 dBi, 8 dBi, 9 dBi, 12 dBi circular polarized antenna
Cable	Low-loss RF cable recommended
Antenna quantity	Depends on 1/4/8/16-port model

For long-distance reading, ask supplier to quote the module together with a **9 dBi or 12 dBi circular polarized antenna**.

7. Certifications to Ask For

For real purchase, ask supplier for:

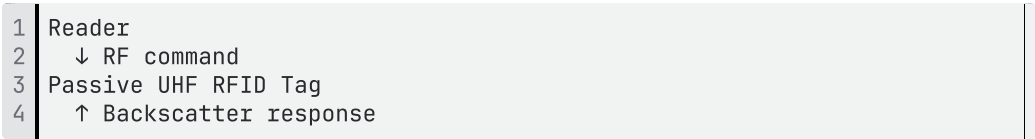
Certification	Why it matters
FCC	Required for US market
CE / RED	Required for EU market
RoHS	Material compliance
SRRC	China RF approval
KC	Korea market
TELEC/MIC	Japan market

Do not assume the bare module has certification. Many low-cost modules only have chip-level or supplier-level documents.

EPC C1G2 / ISO18000-6C / ISO18000-63 Protocol

EPC C1G2, **EPC Gen2**, **ISO18000-6C**, and **ISO/IEC 18000-63** are basically the same UHF RFID passive tag protocol family.

It defines how a UHF RFID reader communicates with passive UHF RFID tags over RF.



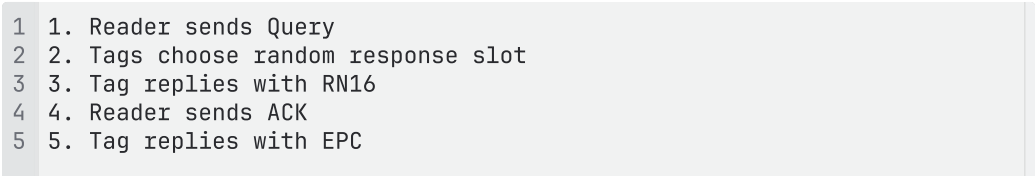
Tag Memory Banks

Bank	Name	Purpose
00	Reserved	Kill password and access password
01	EPC	Main EPC identification code
10	TID	Tag chip/manufacturer ID
11	User	Optional user data

Common Operations

Operation	Purpose
Inventory	Detect tags in antenna field
Read EPC	Read main tag ID
Read TID	Read tag chip ID
Read User Memory	Read custom stored data
Write EPC	Program/change EPC value
Write User Memory	Store application data
Select	Filter target tags
Lock	Lock memory bank
Kill	Permanently disable tag

Inventory Flow



Simplified communication:

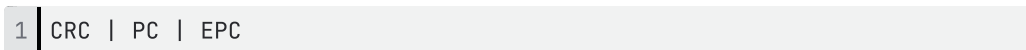


Important Commands

Command	Purpose
Query	Start inventory round
QueryRep	Move to next slot
QueryAdjust	Adjust inventory slot count
ACK	Request EPC from tag
NAK	End tag response
Select	Filter tags
Req_RN	Request tag handle
Read	Read tag memory
Write	Write tag memory
BlockWrite	Write multiple words
Lock	Lock memory/password
Kill	Disable tag

EPC Memory Format

EPC memory usually contains:



Field	Meaning
CRC	Error check
PC	Protocol control bits, including EPC length

EPC	Actual EPC identifier
-----	-----------------------

Example 96-bit EPC:

```
1 300833B2DDD9014000000001
```

24 hex characters × 4 bits = 96 bits.

Practical Integration Behavior

A reader module usually hides the low-level EPC Gen2 RF commands and gives you higher-level API/serial functions like:

```
1 StartInventory()
2 ReadTag()
3 ReadMemory()
4 WriteMemory()
5 SetPower()
6 SetFrequencyRegion()
7 SetAntennaPort()
8 StopInventory()
```

The application usually receives tag data like:

```
1 EPC: 300833B2DDD9014000000001
2 TID: E2801191A503006012345678
3 RSSI: -48 dBm
4 Antenna: 1
5 Frequency: 915.25 MHz
```

Clean Specification Wording

You can write it like this in your own document:

```
1 Supported RFID Protocol:
2 - EPCglobal Class 1 Generation 2
3 - EPC C1G2 / EPC Gen2
4 - ISO18000-6C
5 - ISO/IEC 18000-63
6 - RAIN RFID compatible
7
8 Supported Tag Operations:
9 - Inventory
10 - EPC memory read/write
11 - TID memory read
12 - User memory read/write
13 - Reserved memory access
14 - Select/filter
15 - Lock
16 - Kill
17 - Multi-tag anti-collision
18
19 Supported Tag Memory Banks:
20 - Reserved memory
21 - EPC memory
22 - TID memory
23 - User memory
```


I'll avoid generating supplier-message sections unless you explicitly ask for them.

could you write the code for STM32F407 to implement the protocol?

Yes — but important distinction first: STM32F407 should normally **control the RS2000/R2000 reader module**, not implement the raw EPC Gen2 RF air-interface itself. The RS2000 module handles `Query`, `ACK`, anti-collision, RF modulation, etc.; STM32 talks to the module over UART/RS232/TTL/USB-serial using the module command protocol.

Thought for 4s

Yes. Below is a **STM32F407 HAL UART driver template** for controlling an **RS2000/R2000 UHF RFID reader module**.

Important: this code does **not** implement the raw RF-level EPC Gen2 waveform. The RS2000/R2000 module already does that. This STM32 code implements the **host-side control protocol**:

```
1 | STM32F407 — UART/RS232/TTL — RS2000/R2000 UHF RFID Module — RF —  
   | EPC Gen2 Tags
```

The STM32 sends commands like:

```
1 | Set RF power  
2 | Set frequency region  
3 | Start inventory  
4 | Read EPC  
5 | Read TID  
6 | Read User Memory  
7 | Write EPC/User Memory  
8 | Stop inventory
```

1. Example Project Structure

```
1 | Core/  
2 |   Inc/  
3 |     rs2000.h  
4 |   Src/  
5 |     rs2000.c  
6 |     main.c
```

2. `rs2000.h`

```
1 | #ifndef RS2000_H  
2 | #define RS2000_H  
3 |  
4 | #include "stm32f4xx_hal.h"  
5 | #include <stdint.h>  
6 | #include <stdbool.h>  
7 |  
8 | #define RS2000_MAX_PAYLOAD_LEN    256  
9 | #define RS2000_MAX_FRAME_LEN     300
```

```

10 #define RS2000_MAX_EPC_LEN      64
11 #define RS2000_MAX_TID_LEN      64
12 #define RS2000_UART_TIMEOUT_MS  500
13
14 /*
15  * Generic RS2000/R2000 frame format used in this driver:
16  *
17  * Byte 0: Header 0xA0
18  * Byte 1: Length
19  * Byte 2: Command
20  * Byte 3..N: Payload
21  * Last byte: Checksum
22  *
23  * NOTE:
24  * Many China RS2000/R2000 modules use similar serial packet formats,
25  * but exact command IDs can differ by firmware.
26  * If your module uses different command IDs, update the CMD values
27  * below.
28  */
29 #define RS2000_FRAME_HEADER      0xA0
30
31 typedef enum
32 {
33     RS2000_OK = 0,
34     RS2000_ERROR,
35     RS2000_TIMEOUT,
36     RS2000_INVALID_FRAME,
37     RS2000_CHECKSUM_ERROR,
38     RS2000_BUFFER_TOO_SMALL
39 } RS2000_Status;
40
41 /*
42  * Generic command IDs.
43  * Change these IDs if your module command protocol uses different
44  * values.
45  */
46 typedef enum
47 {
48     RS2000_CMD_GET_VERSION      = 0x01,
49     RS2000_CMD_SET_REGION      = 0x02,
50     RS2000_CMD_SET_RF_POWER    = 0x03,
51     RS2000_CMD_SET_ANTENNA     = 0x04,
52
53     RS2000_CMD_INVENTORY_SINGLE = 0x10,
54     RS2000_CMD_INVENTORY_START = 0x11,
55     RS2000_CMD_INVENTORY_STOP  = 0x12,
56
57     RS2000_CMD_READ_MEMORY     = 0x20,
58     RS2000_CMD_WRITE_MEMORY    = 0x21,
59     RS2000_CMD_LOCK_TAG        = 0x22,
60     RS2000_CMD_KILL_TAG        = 0x23,
61
62     RS2000_CMD_SELECT_TAG      = 0x30
63 } RS2000_Command;
64
65 typedef enum
66 {
67     RS2000_REGION_US           = 0x01,    // 902-928 MHz
68     RS2000_REGION_EU           = 0x02,    // 865-868 MHz
69     RS2000_REGION_CHINA        = 0x03,    // 920-925 MHz
70     RS2000_REGION_KOREA        = 0x04,
71     RS2000_REGION_JAPAN        = 0x05,
72     RS2000_REGION_CUSTOM       = 0xFF
73 } RS2000_Region;
74
75 typedef enum

```

```

75 {
76     RS2000_MEM_RESERVED          = 0x00,
77     RS2000_MEM_EPC               = 0x01,
78     RS2000_MEM_TID               = 0x02,
79     RS2000_MEM_USER              = 0x03
80 } RS2000_MemoryBank;
81
82 typedef struct
83 {
84     UART_HandleTypeDef *huart;
85
86     uint8_t rx_frame[RS2000_MAX_FRAME_LEN];
87     uint16_t rx_len;
88 } RS2000_Handle;
89
90 typedef struct
91 {
92     uint8_t epc[RS2000_MAX_EPC_LEN];
93     uint8_t epc_len;
94
95     uint8_t tid[RS2000_MAX_TID_LEN];
96     uint8_t tid_len;
97
98     int8_t rssi;
99     uint8_t antenna_id;
100 } RS2000_TagInfo;
101
102 void RS2000_Init(RS2000_Handle *dev, UART_HandleTypeDef *huart);
103
104 RS2000_Status RS2000_GetVersion(RS2000_Handle *dev,
105                                 uint8_t *version,
106                                 uint16_t *version_len);
107
108 RS2000_Status RS2000_SetRegion(RS2000_Handle *dev,
109                                RS2000_Region region);
110
111 RS2000_Status RS2000_SetRfPower(RS2000_Handle *dev,
112                                 uint8_t power_dbm);
113
114 RS2000_Status RS2000_SetAntenna(RS2000_Handle *dev,
115                                 uint8_t antenna_id);
116
117 RS2000_Status RS2000_InventorySingle(RS2000_Handle *dev,
118                                      RS2000_TagInfo *tag);
119
120 RS2000_Status RS2000_StartInventory(RS2000_Handle *dev);
121
122 RS2000_Status RS2000_StopInventory(RS2000_Handle *dev);
123
124 RS2000_Status RS2000_ReadMemory(RS2000_Handle *dev,
125                                 const uint8_t *epc,
126                                 uint8_t epc_len,
127                                 RS2000_MemoryBank bank,
128                                 uint16_t word_addr,
129                                 uint16_t word_count,
130                                 uint8_t *out_data,
131                                 uint16_t *out_len);
132
133 RS2000_Status RS2000_WriteMemory(RS2000_Handle *dev,
134                                  const uint8_t *epc,
135                                  uint8_t epc_len,
136                                  RS2000_MemoryBank bank,
137                                  uint16_t word_addr,
138                                  const uint8_t *data,
139                                  uint16_t data_len);
140
141 RS2000_Status RS2000_SelectTagByEpc(RS2000_Handle *dev,

```

```

142                                     const uint8_t *epc,
143                                     uint8_t epc_len);
144
145 #endif

```

3. rs2000.c

```

1  #include "rs2000.h"
2  #include <string.h>
3
4  static uint8_t RS2000_CalcChecksum(const uint8_t *data, uint16_t len)
5  {
6      /*
7       * Generic checksum:
8       * Sum all bytes, take lower 8 bits, then two's complement.
9       *
10     * checksum = 0x100 - (sum & 0xFF)
11     *
12     * Some modules use XOR or simple sum instead.
13     * If your module responds with checksum error, update this
14     function.
15     */
16     uint16_t sum = 0;
17
18     for (uint16_t i = 0; i < len; i++)
19     {
20         sum += data[i];
21     }
22
23     return (uint8_t)(0x100 - (sum & 0xFF));
24 }
25
26 static bool RS2000_VerifyChecksum(const uint8_t *frame, uint16_t len)
27 {
28     if (len < 4)
29     {
30         return false;
31     }
32
33     uint8_t expected = RS2000_CalcChecksum(frame, len - 1);
34     uint8_t actual = frame[len - 1];
35
36     return expected == actual;
37 }
38
39 static RS2000_Status RS2000_BuildFrame(uint8_t cmd,
40                                         const uint8_t *payload,
41                                         uint16_t payload_len,
42                                         uint8_t *out_frame,
43                                         uint16_t *out_len)
44 {
45     if (payload_len > RS2000_MAX_PAYLOAD_LEN)
46     {
47         return RS2000_BUFFER_TOO_SMALL;
48     }
49
50     /*
51     * Frame:
52     * [0] Header
53     * [1] Length = CMD + Payload + Checksum
54     * [2] Command
55     * [3..] Payload
56     * [last] Checksum

```

```

56     */
57     uint16_t index = 0;
58
59     out_frame[index++] = RS2000_FRAME_HEADER;
60     out_frame[index++] = (uint8_t)(payload_len + 2);
61     out_frame[index++] = cmd;
62
63     if (payload != NULL && payload_len > 0)
64     {
65         memcpy(&out_frame[index], payload, payload_len);
66         index += payload_len;
67     }
68
69     out_frame[index] = RS2000_CalcChecksum(out_frame, index);
70     index++;
71
72     *out_len = index;
73
74     return RS2000_OK;
75 }
76
77 static RS2000_Status RS2000_SendCommand(RS2000_Handle *dev,
78                                         uint8_t cmd,
79                                         const uint8_t *payload,
80                                         uint16_t payload_len)
81 {
82     uint8_t tx_frame[RS2000_MAX_FRAME_LEN];
83     uint16_t tx_len = 0;
84
85     RS2000_Status status = RS2000_BuildFrame(cmd,
86                                              payload,
87                                              payload_len,
88                                              tx_frame,
89                                              &tx_len);
90
91     if (status != RS2000_OK)
92     {
93         return status;
94     }
95
96     HAL_StatusTypeDef hal_status = HAL_UART_Transmit(dev->huart,
97                                                      tx_frame,
98                                                      tx_len,
99
100 RS2000_UART_TIMEOUT_MS);
101
102     if (hal_status != HAL_OK)
103     {
104         return RS2000_ERROR;
105     }
106
107     return RS2000_OK;
108 }
109
110 static RS2000_Status RS2000_ReceiveFrame(RS2000_Handle *dev,
111                                           uint8_t *cmd,
112                                           uint8_t *payload,
113                                           uint16_t *payload_len)
114 {
115     uint8_t byte = 0;
116     uint8_t length = 0;
117
118     /*
119      * Wait for header.
120      */
121     do
122     {

```

```

122         if (HAL_UART_Receive(dev->huart,
123                               &byte,
124                               1,
125                               RS2000_UART_TIMEOUT_MS) != HAL_OK)
126         {
127             return RS2000_TIMEOUT;
128         }
129     } while (byte != RS2000_FRAME_HEADER);
130
131     dev->rx_frame[0] = byte;
132
133     /*
134      * Read length.
135      */
136     if (HAL_UART_Receive(dev->huart,
137                           &length,
138                           1,
139                           RS2000_UART_TIMEOUT_MS) != HAL_OK)
140     {
141         return RS2000_TIMEOUT;
142     }
143
144     dev->rx_frame[1] = length;
145
146     /*
147      * Total frame length:
148      * header + length byte + length field
149      */
150     uint16_t total_len = 2 + length;
151
152     if (total_len > RS2000_MAX_FRAME_LEN)
153     {
154         return RS2000_BUFFER_TOO_SMALL;
155     }
156
157     /*
158      * Read remaining bytes: command + payload + checksum.
159      */
160     if (HAL_UART_Receive(dev->huart,
161                           &dev->rx_frame[2],
162                           length,
163                           RS2000_UART_TIMEOUT_MS) != HAL_OK)
164     {
165         return RS2000_TIMEOUT;
166     }
167
168     if (!RS2000_VerifyChecksum(dev->rx_frame, total_len))
169     {
170         return RS2000_CHECKSUM_ERROR;
171     }
172
173     *cmd = dev->rx_frame[2];
174
175     /*
176      * length = command + payload + checksum
177      * payload_len = length - command - checksum
178      */
179     if (length < 2)
180     {
181         return RS2000_INVALID_FRAME;
182     }
183
184     *payload_len = length - 2;
185
186     if (*payload_len > 0)
187     {
188         memcpy(payload, &dev->rx_frame[3], *payload_len);

```

```

189     }
190
191     dev->rx_len = total_len;
192
193     return RS2000_OK;
194 }
195
196 static RS2000_Status RS2000_SendAndReceive(RS2000_Handle *dev,
197                                             uint8_t tx_cmd,
198                                             const uint8_t *tx_payload,
199                                             uint16_t tx_payload_len,
200                                             uint8_t *rx_cmd,
201                                             uint8_t *rx_payload,
202                                             uint16_t *rx_payload_len)
203 {
204     RS2000_Status status;
205
206     status = RS2000_SendCommand(dev,
207                                 tx_cmd,
208                                 tx_payload,
209                                 tx_payload_len);
210
211     if (status != RS2000_OK)
212     {
213         return status;
214     }
215
216     status = RS2000_ReceiveFrame(dev,
217                                  rx_cmd,
218                                  rx_payload,
219                                  rx_payload_len);
220
221     return status;
222 }
223
224 void RS2000_Init(RS2000_Handle *dev, UART_HandleTypeDef *huart)
225 {
226     memset(dev, 0, sizeof(RS2000_Handle));
227     dev->huart = huart;
228 }
229
230 RS2000_Status RS2000_GetVersion(RS2000_Handle *dev,
231                                 uint8_t *version,
232                                 uint16_t *version_len)
233 {
234     uint8_t rx_cmd;
235     uint8_t rx_payload[RS2000_MAX_PAYLOAD_LEN];
236     uint16_t rx_payload_len = 0;
237
238     RS2000_Status status = RS2000_SendAndReceive(dev,
239
240 RS2000_CMD_GET_VERSION,
241
242 NULL,
243 0,
244 &rx_cmd,
245 rx_payload,
246 &rx_payload_len);
247
248 if (status != RS2000_OK)
249 {
250     return status;
251 }
252
253 memcpy(version, rx_payload, rx_payload_len);
254 *version_len = rx_payload_len;
255
256 return RS2000_OK;

```

[illegible]


```

322
323 RS2000_Status RS2000_InventorySingle(RS2000_Handle *dev,
324                                     RS2000_TagInfo *tag)
325 {
326     uint8_t rx_cmd;
327     uint8_t rx_payload[RS2000_MAX_PAYLOAD_LEN];
328     uint16_t rx_payload_len = 0;
329
330     memset(tag, 0, sizeof(RS2000_TagInfo));
331
332     RS2000_Status status = RS2000_SendAndReceive(dev,
333
334     RS2000_CMD_INVENTORY_SINGLE,
335
336     NULL,
337     0,
338     &rx_cmd,
339     rx_payload,
340     &rx_payload_len);
341
342     if (status != RS2000_OK)
343     {
344         return status;
345     }
346
347     /*
348     * Example assumed inventory response payload:
349     *
350     * Byte 0: EPC length
351     * Byte 1..N: EPC
352     * Next byte: RSSI
353     * Next byte: Antenna ID
354     *
355     * Adjust this parser based on your module's real response
356     format.
357     */
358     if (rx_payload_len < 3)
359     {
360         return RS2000_INVALID_FRAME;
361     }
362
363     uint8_t epc_len = rx_payload[0];
364
365     if (epc_len > RS2000_MAX_EPC_LEN)
366     {
367         return RS2000_BUFFER_TOO_SMALL;
368     }
369
370     if ((uint16_t)(1 + epc_len + 2) > rx_payload_len)
371     {
372         return RS2000_INVALID_FRAME;
373     }
374
375     tag->epc_len = epc_len;
376     memcpy(tag->epc, &rx_payload[1], epc_len);
377
378     tag->rss_i = (int8_t)rx_payload[1 + epc_len];
379     tag->antenna_id = rx_payload[1 + epc_len + 1];
380
381     return RS2000_OK;
382 }
383
384 RS2000_Status RS2000_StartInventory(RS2000_Handle *dev)
385 {
386     uint8_t rx_cmd;
387     uint8_t rx_payload[RS2000_MAX_PAYLOAD_LEN];
388     uint16_t rx_payload_len = 0;

```

```

387     return RS2000_SendAndReceive(dev,
388                                   RS2000_CMD_INVENTORY_START,
389                                   NULL,
390                                   0,
391                                   &rx_cmd,
392                                   rx_payload,
393                                   &rx_payload_len);
394 }
395
396 RS2000_Status RS2000_StopInventory(RS2000_Handle *dev)
397 {
398     uint8_t rx_cmd;
399     uint8_t rx_payload[RS2000_MAX_PAYLOAD_LEN];
400     uint16_t rx_payload_len = 0;
401
402     return RS2000_SendAndReceive(dev,
403                                   RS2000_CMD_INVENTORY_STOP,
404                                   NULL,
405                                   0,
406                                   &rx_cmd,
407                                   rx_payload,
408                                   &rx_payload_len);
409 }
410
411 RS2000_Status RS2000_SelectTagByEpc(RS2000_Handle *dev,
412                                     const uint8_t *epc,
413                                     uint8_t epc_len)
414 {
415     /*
416      * Generic Select payload:
417      *
418      * Byte 0: Memory bank = EPC
419      * Byte 1: EPC length in bytes
420      * Byte 2..N: EPC data
421      */
422     if (epc_len > RS2000_MAX_EPC_LEN)
423     {
424         return RS2000_BUFFER_TOO_SMALL;
425     }
426
427     uint8_t payload[RS2000_MAX_EPC_LEN + 2];
428
429     payload[0] = RS2000_MEM_EPC;
430     payload[1] = epc_len;
431     memcpy(&payload[2], epc, epc_len);
432
433     uint8_t rx_cmd;
434     uint8_t rx_payload[RS2000_MAX_PAYLOAD_LEN];
435     uint16_t rx_payload_len = 0;
436
437     return RS2000_SendAndReceive(dev,
438                                   RS2000_CMD_SELECT_TAG,
439                                   payload,
440                                   epc_len + 2,
441                                   &rx_cmd,
442                                   rx_payload,
443                                   &rx_payload_len);
444 }
445
446 RS2000_Status RS2000_ReadMemory(RS2000_Handle *dev,
447                                  const uint8_t *epc,
448                                  uint8_t epc_len,
449                                  RS2000_MemoryBank bank,
450                                  uint16_t word_addr,
451                                  uint16_t word_count,
452                                  uint8_t *out_data,
453                                  uint16_t *out_len)

```

```

454 {
455     /*
456     * Generic read payload:
457     *
458     * Byte 0: EPC length
459     * Byte 1..N: EPC
460     * Next byte: Memory bank
461     * Next 2 bytes: Word address, big endian
462     * Next 2 bytes: Word count, big endian
463     *
464     * Word = 16 bits.
465     */
466     if (epc_len > RS2000_MAX_EPC_LEN)
467     {
468         return RS2000_BUFFER_TOO_SMALL;
469     }
470
471     uint8_t payload[RS2000_MAX_EPC_LEN + 6];
472     uint16_t index = 0;
473
474     payload[index++] = epc_len;
475
476     memcpy(&payload[index], epc, epc_len);
477     index += epc_len;
478
479     payload[index++] = (uint8_t)bank;
480
481     payload[index++] = (uint8_t)((word_addr >> 8) & 0xFF);
482     payload[index++] = (uint8_t)(word_addr & 0xFF);
483
484     payload[index++] = (uint8_t)((word_count >> 8) & 0xFF);
485     payload[index++] = (uint8_t)(word_count & 0xFF);
486
487     uint8_t rx_cmd;
488     uint8_t rx_payload[RS2000_MAX_PAYLOAD_LEN];
489     uint16_t rx_payload_len = 0;
490
491     RS2000_Status status = RS2000_SendAndReceive(dev,
492     RS2000_CMD_READ_MEMORY,
493
494                                     payload,
495                                     index,
496                                     &rx_cmd,
497                                     rx_payload,
498                                     &rx_payload_len);
499
500     if (status != RS2000_OK)
501     {
502         return status;
503     }
504
505     memcpy(out_data, rx_payload, rx_payload_len);
506     *out_len = rx_payload_len;
507
508     return RS2000_OK;
509 }
510
511 RS2000_Status RS2000_WriteMemory(RS2000_Handle *dev,
512                                 const uint8_t *epc,
513                                 uint8_t epc_len,
514                                 RS2000_MemoryBank bank,
515                                 uint16_t word_addr,
516                                 const uint8_t *data,
517                                 uint16_t data_len)
518 {
519     /*
520     * Generic write payload:

```

```

520      *
521      * Byte 0: EPC length
522      * Byte 1..N: EPC
523      * Next byte: Memory bank
524      * Next 2 bytes: Word address, big endian
525      * Next 2 bytes: Data length in bytes, big endian
526      * Next bytes: Data
527      *
528      * Data length should usually be even because EPC Gen2 memory is
word-based.
529      */
530      if (epc_len > RS2000_MAX_EPC_LEN)
531      {
532          return RS2000_BUFFER_TOO_SMALL;
533      }
534
535      if ((RS2000_MAX_EPC_LEN + data_len + 8) > RS2000_MAX_PAYLOAD_LEN)
536      {
537          return RS2000_BUFFER_TOO_SMALL;
538      }
539
540      uint8_t payload[RS2000_MAX_PAYLOAD_LEN];
541      uint16_t index = 0;
542
543      payload[index++] = epc_len;
544
545      memcpy(&payload[index], epc, epc_len);
546      index += epc_len;
547
548      payload[index++] = (uint8_t)bank;
549
550      payload[index++] = (uint8_t)((word_addr >> 8) & 0xFF);
551      payload[index++] = (uint8_t)(word_addr & 0xFF);
552
553      payload[index++] = (uint8_t)((data_len >> 8) & 0xFF);
554      payload[index++] = (uint8_t)(data_len & 0xFF);
555
556      memcpy(&payload[index], data, data_len);
557      index += data_len;
558
559      uint8_t rx_cmd;
560      uint8_t rx_payload[RS2000_MAX_PAYLOAD_LEN];
561      uint16_t rx_payload_len = 0;
562
563      return RS2000_SendAndReceive(dev,
564                                  RS2000_CMD_WRITE_MEMORY,
565                                  payload,
566                                  index,
567                                  &rx_cmd,
568                                  rx_payload,
569                                  &rx_payload_len);
570 }

```

4. Example `main.c` Usage

This assumes your STM32CubeMX generated:

```
1 | UART_HandleTypeDef huart1;
```

Use USART1 or whichever UART is connected to the RS2000 module.

```
1 | #include "main.h"
```

```

2 #include "rs2000.h"
3 #include <string.h>
4 #include <stdio.h>
5
6 extern UART_HandleTypeDef huart1;
7
8 RS2000_Handle rfid;
9
10 static void PrintHex(const uint8_t *data, uint16_t len)
11 {
12     for (uint16_t i = 0; i < len; i++)
13     {
14         printf("%02X", data[i]);
15     }
16     printf("\r\n");
17 }
18
19 int main(void)
20 {
21     HAL_Init();
22     SystemClock_Config();
23
24     MX_GPIO_Init();
25     MX_USART1_UART_Init();
26
27     RS2000_Init(&rfid, &huart1);
28
29     HAL_Delay(500);
30
31     /*
32      * Configure module.
33      */
34     RS2000_SetRegion(&rfid, RS2000_REGION_US);
35     HAL_Delay(100);
36
37     RS2000_SetRfPower(&rfid, 30);
38     HAL_Delay(100);
39
40     RS2000_SetAntenna(&rfid, 1);
41     HAL_Delay(100);
42
43     while (1)
44     {
45         RS2000_TagInfo tag;
46
47         RS2000_Status status = RS2000_InventorySingle(&rfid, &tag);
48
49         if (status == RS2000_OK)
50         {
51             printf("Tag found\r\n");
52
53             printf("EPC: ");
54             PrintHex(tag.epc, tag.epc_len);
55
56             printf("RSSI: %d dBm\r\n", tag.rssi);
57             printf("Antenna: %u\r\n", tag.antenna_id);
58
59             /*
60              * Example: read TID memory.
61              * TID usually starts at word address 0.
62              */
63             uint8_t tid_data[64];
64             uint16_t tid_len = 0;
65
66             status = RS2000_ReadMemory(&rfid,
67                                     tag.epc,
68                                     tag.epc_len,

```

```

69         RS2000_MEM_TID,
70         0x0000,
71         6,
72         tid_data,
73         &tid_len);
74
75     if (status == RS2000_OK)
76     {
77         printf("TID: ");
78         PrintHex(tid_data, tid_len);
79     }
80 }
81 else
82 {
83     printf("No tag or reader error: %d\r\n", status);
84 }
85
86     HAL_Delay(500);
87 }
88 }

```

5. UART Configuration

Typical RS2000/R2000 modules use one of these UART settings:

```

1 Baud rate: 115200
2 Data bits: 8
3 Parity: None
4 Stop bits: 1
5 Flow control: None

```

STM32CubeMX setting:

```

1 USARTx Mode: Asynchronous
2 Baud Rate: 115200
3 Word Length: 8 Bits
4 Parity: None
5 Stop Bits: 1
6 Hardware Flow Control: None

```

6. Hardware Connection

For TTL UART module:

```

1 STM32F407 TX → RS2000 RX
2 STM32F407 RX ← RS2000 TX
3 STM32F407 GND ↔ RS2000 GND

```

Power depends on module:

```

1 Small module: often 5V
2 Industrial module: often 9V or 12V
3 Logic level: often 3.3V TTL or 5V TTL

```

Be careful here: **RS232 is not the same as TTL UART.**

If the RS2000 module has **RS232**, use a level converter:

```
1 | STM32 UART TTL → MAX3232 → RS2000 RS232
```

If the module has **TTL UART**, connect directly only if voltage levels match.

7. How EPC Gen2 Operations Map to This Code

Inventory

Application call:

```
1 | RS2000_InventorySingle(&rfid, &tag);
```

Module internally performs:

```
1 | Query
2 | RN16
3 | ACK
4 | EPC response
5 | Anti-collision
```

STM32 only receives the final EPC result.

Read TID

Application call:

```
1 | RS2000_ReadMemory(&rfid,
2 |                   tag.epc,
3 |                   tag.epc_len,
4 |                   RS2000_MEM_TID,
5 |                   0x0000,
6 |                   6,
7 |                   tid_data,
8 |                   &tid_len);
```

Memory bank:

```
1 | TID = bank 10b = 0x02
```

Read User Memory

```
1 | RS2000_ReadMemory(&rfid,
2 |                   tag.epc,
3 |                   tag.epc_len,
4 |                   RS2000_MEM_USER,
5 |                   0x0000,
6 |                   8,
7 |                   user_data,
8 |                   &user_len);
```

Write User Memory

```
1 uint8_t data_to_write[] = {
2     0x12, 0x34,
3     0x56, 0x78
4 };
5
6 RS2000_WriteMemory(&rfid,
7                     tag.epc,
8                     tag.epc_len,
9                     RS2000_MEM_USER,
10                    0x0000,
11                    data_to_write,
12                    sizeof(data_to_write));
```

Because EPC Gen2 memory is word-based, write data should usually be an even number of bytes.

8. EPC C1G2 Memory Bank Values

Use this in your firmware:

```
1 #define EPC_BANK_RESERVED  0x00
2 #define EPC_BANK_EPC        0x01
3 #define EPC_BANK_TID        0x02
4 #define EPC_BANK_USER       0x03
```

Meaning:

```
1 00 = Reserved memory
2 01 = EPC memory
3 10 = TID memory
4 11 = User memory
```

9. What You May Need to Modify

The most likely parts you need to change are these:

```
1 typedef enum
2 {
3     RS2000_CMD_GET_VERSION      = 0x01,
4     RS2000_CMD_SET_REGION      = 0x02,
5     RS2000_CMD_SET_RF_POWER    = 0x03,
6     RS2000_CMD_SET_ANTENNA     = 0x04,
7     RS2000_CMD_INVENTORY_SINGLE = 0x10,
8     RS2000_CMD_READ_MEMORY     = 0x20,
9     RS2000_CMD_WRITE_MEMORY    = 0x21
10 } RS2000_Command;
```

and maybe this:

```
1 static uint8_t RS2000_CalcChecksum(const uint8_t *data, uint16_t len)
```

because different RS2000/R2000 vendors use slightly different serial packet formats.

But the upper-level application logic remains the same:


```
1 Initialize UART
2 Set region
3 Set power
4 Set antenna
5 Inventory tag
6 Read EPC/TID/User
7 Write EPC/User
```

10. Minimal Integration Flow

For your STM32F407 firmware, the clean workflow is:

```
1 1. Boot STM32
2 2. Initialize UART
3 3. Initialize RS2000 driver
4 4. Set frequency region
5 5. Set RF power
6 6. Select antenna
7 7. Run inventory
8 8. Parse EPC result
9 9. Optionally read TID/User memory
10 10. Send result to display, server, CAN, RS485, Ethernet, etc.
```

This is the right architecture:

```
1 Application Layer
2   ↓
3 RS2000 Driver Layer
4   ↓
5 UART / RS232 / TTL
6   ↓
7 RS2000/R2000 Reader Module
8   ↓
9 EPC C162 / ISO18000-6C RF Protocol
10  ↓
11 UHF RFID Tag
```